



6CCS3PRJ Final Year
Detection of Hate Speech towards the
LGBTQIA+ community using
lexicon-based sentiment analysis

Final Project Report

Author: Benjamin Ward

Supervisor: Dr Leonardo Magela Cunha

Student ID: 20036800

April 6, 2023

Abstract

On the internet, people can express their opinions so easily whilst reaching so many. However, not everything that is said, should be said. An example of this is hate speech, this type of content should not be spread online. This paper will compare different methods to stop the spread of hate speech targeted towards the LGBTQIA+ community. The method chosen to implement will be a lexicon-based method for the detection of LGBTQIA+ related hate speech. This tool could then be used for the detection of these negative comments. The tool will be designed in a modular architecture, centred around a controller. The implementation will then be discussed, followed by the analysis of the results. The accuracy of the system was 49% for the detection of LGBTQIA+ hate speech. However, all of the LGBTQIA+ related comments were identified as being LGBTQIA+ related. In conclusion, the system shows promise with future development, however, it is not recommended for use in its current state.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Benjamin Ward

April 6, 2023

Acknowledgements

I would like to thank my supervisor for this project, Dr Leonardo Magela Cunha. I appreciate all of the support and guidance you provided throughout the project. Our meetings were a valuable time to reflect on the progress made and plan for the future.

Contents

1	Introduction	3
1.1	Aims and Objectives	3
1.2	Structure	3
2	Background	5
2.1	Context	5
2.2	Prevention Methods	6
2.3	Detection Methods	7
3	Requirements	10
3.1	Functional Requirements	10
3.2	Non-Functional Requirements	10
3.3	User Requirements	11
4	Specification	12
5	Design	15
5.1	Use Case Diagram	16
5.2	UML Component Diagram	16
5.3	Class Diagram	17
6	Implementation	22
6.1	Text parser	23
6.2	Reformatting SentiWordNet	24
6.3	Hate Speech Detection	24
6.4	LGBTQIA+ Detection	26
6.5	The Controller	26
6.6	Analysis of Test Data	26
6.7	Interface	28
6.8	Additional Features	28
6.9	Testing	29
7	Legal, Social, Ethical and Professional Issues	30
7.1	Different English speaking cultures	30
7.2	Miss-classification	30
7.3	British Computer Society's Code of Conduct	31

8	Results and Evaluation	32
8.1	Evaluation	32
8.2	Results	32
8.3	Related Word	36
9	Conclusion and Future Work	37
9.1	Conclusion	37
9.2	Future Work	38
	Bibliography	41
A	Extra Information	42
A.1	Tables, proofs, graphs, test cases,	42
B	Source Code	43
B.1	Instructions	43

Chapter 1

Introduction

The report is about the creation of a tool to detect hate speech targeted towards the LGBTQIA+ community. This tool can then be later implemented into other systems to detect and then remove hate speech online.

1.1 Aims and Objectives

The tool should be able to detect hate speech targeted towards the LGBTQIA+ community. The system will focus on classifying text that is in the English language. The tool should also be able to work with many different formats of text so that the user can implement it where they desire. The development of the program should be made so that it is easy to update the system for the detection of hate speech from other groups of people.

1.2 Structure

The report will start with the 'Background', this will add context for the creation of the system and why the system is using a particular method compared to other methods.

The 'Requirements' will outline the requirements that have been set for the system to achieve. The requirements are grouped into 'Functional', 'Non-Functional' and 'User' requirements.

The 'Specification' will give more detail for each requirement whilst ordering the requirements into priority.

Next will be the 'Design', which will discuss the reasons for the design of the system. There will be a 'Use Case Diagram', 'UML Component Diagram' and a 'Class Diagram'.

The 'Implementation' chapter will discuss details about the development process of the system. This will be done chronologically to explain the system and the way it was created.

The next chapter is the 'Legal, Social, Ethical and Professional Issues'. This will discuss any potential issues with the project that should be made aware of.

The 'Results and Evaluation' will examine the results of the system with some test data. Some examples will be given to show a breakdown of how some results were made. These results of the tool will then be compared to other implementations of similar systems others have developed.

Finally, there will be the 'Conclusion and Future Work' chapter. The conclusion will discuss what was learnt throughout the process of creating the tool as well as an honest opinion of the final system. Lastly, the potential future work that could improve the system will be discussed.

Chapter 2

Background

This section will provide context for the paper and why it is important to research this topic and decide what solution is best. It will also highlight the different types of prevention methods that could be used and then come to a conclusion as to what approach is best suited for this situation. Then it will go over the two main methods, the positives and negatives attributed to them and review an example for each. Finally, a conclusion will be reached to determine the approach that is best fit for the research question.

2.1 Context

In modern society everyone is connected to the internet, whether you are online to watch movies, play games, or work. This means that people are online and connected consistently. For example, there has been much concern with children consuming all the content given to them since they have been born into the age of the internet and thus exposed to so much content.[7] There are dedicated channels for children to use which have more regulated and safer content however these types of options are not always regularly available to use. When this happens, this allows kids to be exposed to the less regulated side of the internet. Most of the frequently used websites would have some general regulations to moderate what is allowed and what is not allowed. But due to the vast size of many websites, it becomes a difficult task to not let anything unwanted slip through the cracks.[20]

These concerns with the internet do not end with children, everyone can be at risk of being exposed to unwanted harmful content. The list of topics you would want to suppress would be endless but I will be focusing on hate speech. Hate speech is defined by Oxford Advanced

Learner’s Dictionary as ”speech or writing that attacks or threatens a particular group of people, especially on the basis of race, religion or sexual orientation”.[9] This paper will focus on specifically hate speech targeted toward the LGBTQIA+ community. If a person is exposed to these kinds of comments, they could lead to two unwanted outcomes. First, the person might adopt these views and then spread the hate to more people. The other outcome is that the comments could be hurtful to the victim. This gives the developers of these sites a serious responsibility to protect people from these situations. Therefore, this paper will attempt to answer the question ’How can you protect minority groups from online hate speech?’.

2.2 Prevention Methods

There are a few methods that could be used to protect users of the internet from hate speech and they will be examined next.

2.2.1 Education

The first one to mention is education. If resources were put into education so people understand that harmful comments are a serious offence and that the comments can do serious damage to victims, then the publishers of the hate speech could reconsider their actions and even prevent future comments from being created. The benefit of this approach is that it can stop the creation of hate speech thus, it cannot do any harm. The negative however would be that this method might not be very effective, this is because some people might post hate speech with the intent of harm and have a full understanding of their actions. This means that their behaviour might not change.

2.2.2 Moderation and Removal

The next method to consider would be moderation and removal. This would consist of controlling what can be posted online as well as removing any existing harmful comments. The benefit would be that the hateful content can be removed when uploaded or even prevented from being uploaded in the first place. The negatives are that it could cost a lot of labour to check all the content being uploaded. Also, removing content after it has been uploaded means that there was a window of opportunity after the time of upload and the time of removal in which harm could have been caused.

2.2.3 Support

The final method that would bring benefits would be supporting the victims. This method is less of a prevention of hate speech and more of a contingency plan when it does happen. The benefit of this would be reducing the impact hate speech has on victims. The negative of this is that it relies on people already being exposed and of course, this is not ideal.

2.2.4 Conclusion

In an ideal situation, these three methods should all be used to counter hate speech. This paper will focus on the moderation/removal of hate speech. This is because the negative of needing a high labour force to filter the content can be largely automated with programming. The other negative of there still being a window of opportunity to create harm can also be mitigated by examining all the comments at upload time. The first step of moderation/removal will be detection. You must identify what is considered hate speech towards the LGBTQIA+ community and what is not before you decide what course of action to take. There are many different approaches to do this and they will be discussed next.

2.3 Detection Methods

2.3.1 Machine learning

One method is a Machine learning approach. Machine learning is a type of artificial intelligence that identifies patterns in data it has been given and with the recognized patterns, it can classify new unseen examples.[15] Machine learning accuracy depends on the context in which it is being used however, it has been used many times for classification problems which would be what a hate speech detection towards the LGBTQIA+ would need. There are different types of machine learning techniques but many popular methods need training data. This data is analyzed by the algorithm to improve the accuracy of the program. This means that having access to large amounts of this data can be beneficial for creating an accurate algorithm. The problem is that if you do not have this training data, the accuracy can suffer greatly. This means that it could be a challenge to create a hate speech detection system for groups of people that do receive hate speech however, not enough examples to train the algorithm with. Also, depending on the implementation of a machine learning algorithm, it could be challenging for the program to give a 'reason' for its decision.[19] This issue is more important depending on the context and the use of the algorithm. Using the algorithm more like a black box, that

is giving an input and then an output is created without any knowledge of how the output was created. This type of decision-making might not be acceptable in more sensitive systems. 'Detecting Harmful Online Conversational Content towards LGBTQIA+ Individuals'[8] used two out-of-the-box machine learning algorithms to detect harmful content toward LGBTQIA+ individuals. This is a demonstration that even despite the difficulties that come with machine learning, they provided good comparisons between the different models and the results show that the different models can be effective in detecting harmful content. However, due to their focus on evaluating the results of models, it would have been more appropriate to train and compare the models on a higher variety of datasets to reduce biases rather than using one. Despite this criticism, it must also be taken into consideration that a variety of datasets on the LGBTQIA+ community could be hard to acquire due to the topic.

2.3.2 Lexicon-based Analysis

The lexicon-based approach works by breaking up the text into individual words and then identifying the score related to each word. This score will represent the positivity or negativity the word has. The next phase is to then use the collection of these scores to represent the text as a whole.[5] A positive of this approach is that it is a general way to classify any type of text. This means that you can classify different texts even when they have different/unique formats. Another positive is that the approach is very understandable for users and developers to understand the inner workings. This means that if a given text has been classified in a certain way, it would be possible to extract the reasons behind this e.g. the words 'A' and 'B' have scores 'x' and 'y'. Thus the sum for the text is 'x' + 'y'. However, a negative of this approach is that by analysing the words separately, you remove the context that the word was used in compared to the other words. This can change the meaning of words and the overall text. However, 'Lexicon-Based Sentiment Analysis in the Social Web'[17] used this method to classify Twitter messages to identify how positive or negative they were. The implementation created a high accuracy for both binary and multi-class classification of Twitter messages. However, if they used more than one dataset when testing the data, this could have created a more realistic representation of the results due to negating any hidden biases. Instead, they relied on one dataset, hoping that no biases would be contained.

2.3.3 Conclusion

Both methods of approach have some interesting benefits, but the lexicon-based approach will be used in the context of hate speech detection towards the LGBTQIA+ community. This is because it can be used on many different types of text, even unseen formats. These new formats would be highly beneficial to classify for the same reasons as to classify other texts. However, a machine learning approach might have trouble classifying a format it has not seen before, thus causing misclassification to occur. This leads to the second reason why lexicon-based analysis over machine learning, is the topic of misclassification and explainable decisions. Due to the sensitivity of the use of the algorithm, it would be desirable to have a highly accurate program to never misclassify text. However, in the situation that some text has been classified as hate speech towards LGBTQIA+, the owner of the texts should be allowed to appeal their case that it is misclassified. This means that having an algorithm that can explain the decision process is desirable and machine learning can have trouble doing this, unlike lexicon-based analysis.

Chapter 3

Requirements

The purpose of the software is to create a hate speech detection system that can target hate speech towards the LGBTQIA+ community. The requirements are grouped into subsections, 'Functional Requirements', 'Non-Functional Requirements' and 'User Requirements'.

3.1 Functional Requirements

- 1.1 The system is able to detect if the text contains hate speech.
- 1.2 The system is able to detect if the text is targeted towards the LGBTQIA+ community.
- 1.3 The system is able to manage any format of text.
- 1.4 The system is able to identify if a word is positive, negative or neutral.
- 1.5 The system is able to identify if a word indicates that the text is targeted towards the LGBTQIA+ community.
- 1.6 The system is able to extract any emojis from a given text.

3.2 Non-Functional Requirements

- 2.1 The system is able to process text and generate a response within a short and undistruptive time frame.
- 2.2 The system is designed and implemented in a modular way so different sections can be swapped out with little effort.
- 2.3 The system is able to be used by other programs easily through an interface.

3.3 User Requirements

- 3.1** The user is able to input any given text and be returned the result if there is a presence of hate speech towards the LGBTQIA+ community.
- 3.2** The user is able to input a list of texts and be returned a list of results if there is a presence of hate speech towards the LGBTQIA+ community within the separate texts.
- 3.3** The user is able to toggle whether hate speech detection is on or not.
- 3.4** The user is able to toggle whether LGBTQIA+ related text detection is on or not.
- 3.5** The user is able to change the threshold for what is considered hate speech.
- 3.6** The user is able to change the threshold for what is considered positive text.
- 3.7** The user is able to change the threshold that a word needs to reach to be considered a positive or negative word.
- 3.8** The user is able to compute a desired number of demo texts to be tested on with the current settings.
- 3.9** The user is able to get the results of the demo texts with the given settings.
- 3.10** The user is able to reset the demo test results so new settings can be tested.
- 3.11** The user is able to view a list of all the words and their positivity/negativity values.
- 3.12** The user is able to add a new word and the corresponding positivity/negativity value.
- 3.13** The user is able to remove any given word and the corresponding positivity/negativity value.
- 3.14** The user is able to reset positivity/negativity values to the original values.
- 3.15** The user is able to view a list of all the LGBTQIA+ related words.
- 3.16** The user is able to add a new word to the list of LGBTQIA+ related words.
- 3.17** The user is able to remove a word from the list of LGBTQIA+ related words.
- 3.18** The user is able to reset the LGBTQIA+ related words to the default list of words.
- 3.19** The user is able to save the current state's settings.
- 3.20** The user is able to load previously saved settings.

Chapter 4

Specification

This next section will cover the specifications of the requirements. This will include the ID number to identify the requirement and the specifications for the requirement. These specifications will be sectioned in priority.

High Priority

- 1.1** The system will calculate a score that will represent the text as a whole. This will be done by using the words individual scores. The score, if reaching a certain threshold, will be considered as hate speech, positive text or neutral.
- 1.2** The system will detect if any of the individual words indicate that the subject of the text is the LGBTQIA+ community.
- 1.4** The system will obtain the different scores for the given word, and then produce one score to represent the word. This will require a database of words that map a word to a positivity/negativity score.
- 1.5** The system will cross reference the word to other words that are known to indicate the subject being related to the LGBTQIA+ community. This will require a database of LGBTQIA+ related words.
- 1.6** The system will be able to identify and locate the presence of any of the most popular emojis from the text. This will require a database of commonly used emojis.
- 3.1** The system will determine if the text is considered hate speech or not. The system will also consider if the text is aimed towards the LGBTQIA+ community or not. Both

processes combined will determine the outcome.

- 3.2** The system will analyse the texts one at a time, producing a result that will be later returned when all the analysis is done for all the texts.

Medium Priority

- 1.3** The system will identify the words within the text and separate them from the rest of the text. These separate words will all be normalized to the same style to allow consistency, allowing future analyses to make assumptions about the text.
- 2.3** There will be one main access point where other programs will interact with the system. This will reduce the complexity of the system when being used.
- 3.5, 3.6** This will give the user more control over the sensitivity of what is considered hate speech and what is classified as positive text. This allows control over the intensity of hate speech or positive text they would like to detect.
- 3.7** This will give the user control over what words are considered positive and what are negative. These fine-grain changes will propagate the value for the whole text, possibly changing the classification.
- 3.8** This provides the user will full control over how many demo texts will be classified. Providing control over the computational power designated to the testing. The results of the tests will be stored. The run of future tests will use new demo texts and the results will be stored alongside old results, building up the number of results each time.
- 3.9** This will provide the user with statistics of the testing such as the true positive rate of classification and the rate of hate speech detection towards the LGBTQIA+ community was successful.
- 3.10** This will remove all the results previously calculated. This will allow the user to be able to test the differences in results when settings are changed.

Low Priority

- 2.1** The performance of the system is a factor for any user of the system. This is because the system is designed as a tool for users to implement and use within their own projects. The

performance of the system will have a direct impact on the performance of the system using it.

2.2 The modular design will allow maintenance and modification of the system to be easy. This will allow further development on the system such as adding detection for other minority groups to be easily done without much change to the rest of the system.

3.3 This allows the user to disassociate the text from hate speech detection, and instead focus the algorithms on the detection of LGBTQIA+ targeted content.

3.4 This allows the user to disassociate the text from the detection of LGBTQIA+ targeted content, and instead focus the algorithms on the detection of hate speech.

3.11, 3.12, 3.13, 3.14 This gives the user fine-grained customizability on the internals of the system related to hate speech detection. These changes can affect the classification of texts in relation to hate speech.

3.15, 3.16, 3.17, 3.18 This gives the user the ability to change what LGBTQIA+ related words considered in the system. This will allow the user to be more specific with what they would like to detect. These options can give the user control to be able to detect other minority groups by adding related words.

3.19, 3.20 This will allow the user to create different versions of the preferred settings. These saved states would give the user quick access to the desired presets quickly.

Chapter 5

Design

The systems design is an important factor because it can have a large impact on the implementation, maintainability and efficiency of the system. With this in mind, many design patterns must be taken into consideration first before deciding on one to follow.

The factory design pattern[16] uses abstract classes that on their own, do not do much. However, other classes inherit from the abstract class to then inherit the parts of the class that they desire to use. This architecture is useful when dealing with multiple classes that have similarities with one another. However, due to the requirements of the system, many of the components will have different responsibilities therefore, the factory method would not be sufficient.

Another architecture is the Model-View-Controller (MVC)[24]. Following this means that the system will be represented in three sections, the Model, the View and the Controller. The Model will control and maintain the data. This section would fit into the system well due to the dependency the program will have on databases. The next section is the View, here is where all the interface displayed to the user is managed. There will be interaction with the user in the system however, due to the system being used as a tool, an actual complex interface is not needed. The last section is the Controller, the idea of the controller is to connect the Model and View together without direct communication. Due to the requirements of the system, the MVC architecture is not suited however, the idea of using a Controller to manage and relay information from one section to another would make a good component.

Finally, a modular design will be analysed.[22] The concept of modularity is to break down the system into sections that are responsible for a particular part of the system. This means that one module can be replaced by another module with a different implementation however, the rest of the system will not need to change how they operate. An advantage of this would be easy maintainability if one section of the system needs to be replaced, it can with little effect on the rest of the system. Also, it helps with readability because if a developer was analysing the code, due to each module having their own responsibilities, the developer will instantly know what they are reading about.

In conclusion, the design of the system will follow a modular design approach with a Controller to manage modules.

5.1 Use Case Diagram

The use case diagram 5.1 shows all of the ways the user can interact with the system. There are many different possibilities because the system is designed as a tool for other programs to use. This means that the more customizability and options you can provide will allow the user to work with the system that best meets their requirements.

5.2 UML Component Diagram

The diagram 5.2 displays the system as a 4-tiered model. Each box represents different components within the program. The connections between the components represent that one of the components provides certain methods and the other component requires those methods. For example, the TextParser provides a service and the Controller and the HateSpeechDetector requires those methods.

Each tier categorises the components of the system, this helps organise the system into relatable sections whilst keeping a modular design. The top tier depicted is the client tier. This contains the ways a user can interact. In this case, the Interface component is the only interaction point for a user. This is done so that the user can use the system from a single component to maintaining usability. The next tier is the business tier and this tier is about the business logic, which will control how the system works together to produce the services provided. The Controller component has many connections to it, this is because this is the

centre of the business logic of the system. This is done so there is a centralised point of operations for the system, allowing easy maintainability and readability of the program. The third tier is the Integration tier and this tier acts as the bridge between the business tier and the resource tier. This allows the components within the business tier to not need to know about where the data comes from and how it is processed but instead, the Integration tier will do that and reformat any of the data in a specific way, ready for being used. This leads to the final tier, the Resource tier. This tier consists of all of the raw data the system requires. These components are a key section of the system because so much of the business logic is dependent on having access to this information.

An anomaly within the diagram is the component Initializer. This is because this component is not required by any other part of the system. This is because this component was responsible for reformatting SentiWordNet.txt into a more desirable state called 'SentiWordNet.csv'.

5.3 Class Diagram

The diagrams 5.3 and 5.4 represent the classes in the system. Class diagrams usually display the variable a class has, the methods you can call as well as the connections between the classes themselves. These diagrams do display the variables and methods, but not the connections. This was done because the connections between the classes are depicted in 5.2 and if they were shown in the class diagrams then the diagrams themselves would be hard to read due to all the information being presented at once. In 5.3 it can be noticed that the Interface has many methods, due to all of the interactions a user can have with the system however, there are not many variables the Interface holds. This is because the Interface class has the responsibility to pass on the requests to the relevant class instead of computing the requests. The Controller class is also large, this is because it must hold a reference to many of the other classes so they can be managed. In 5.4 the HateSpeechDetector class is also large, this is because the computation of the hate speech detection is a complex task that requires multiple steps.

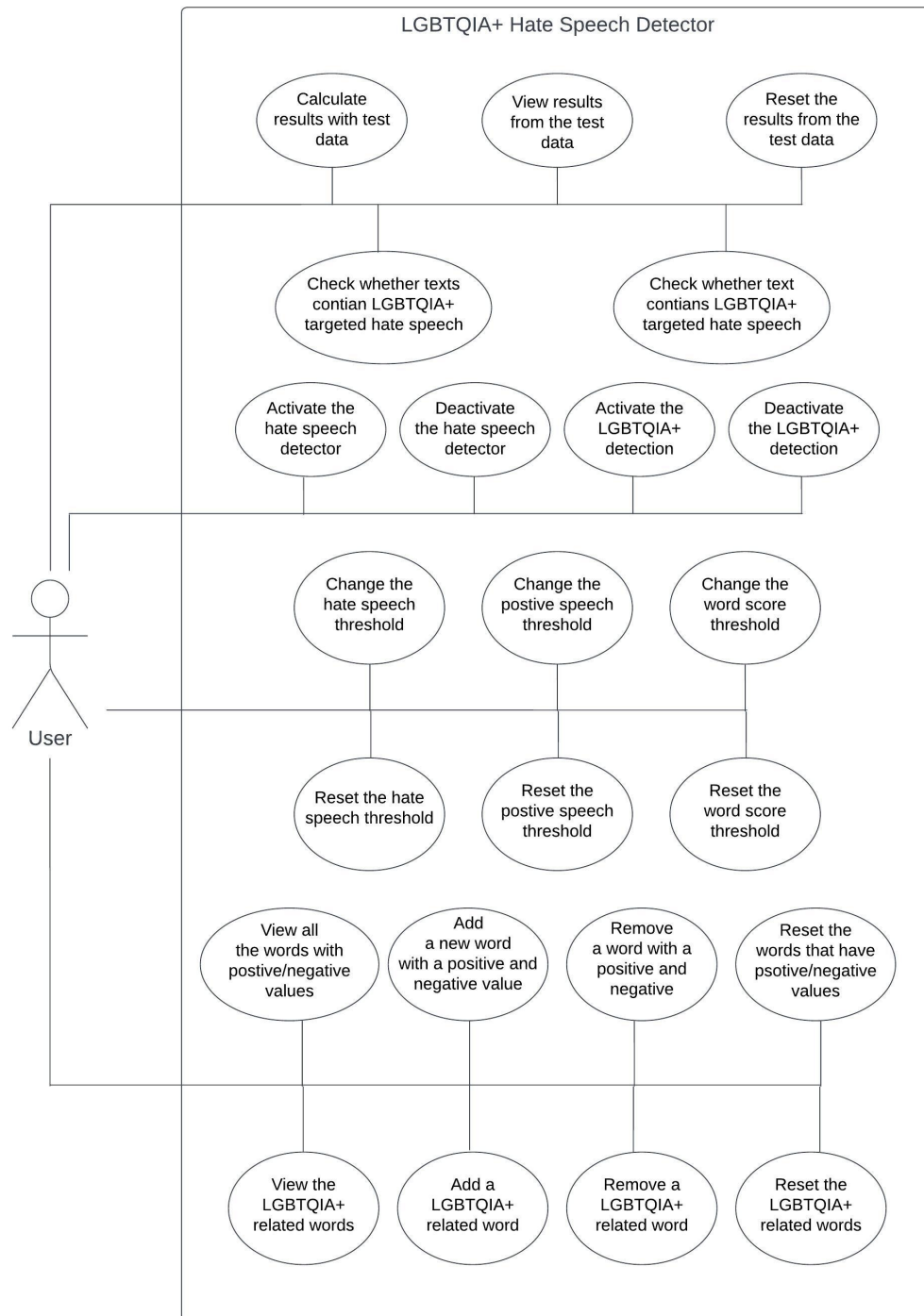


Figure 5.1: Use Case Diagram

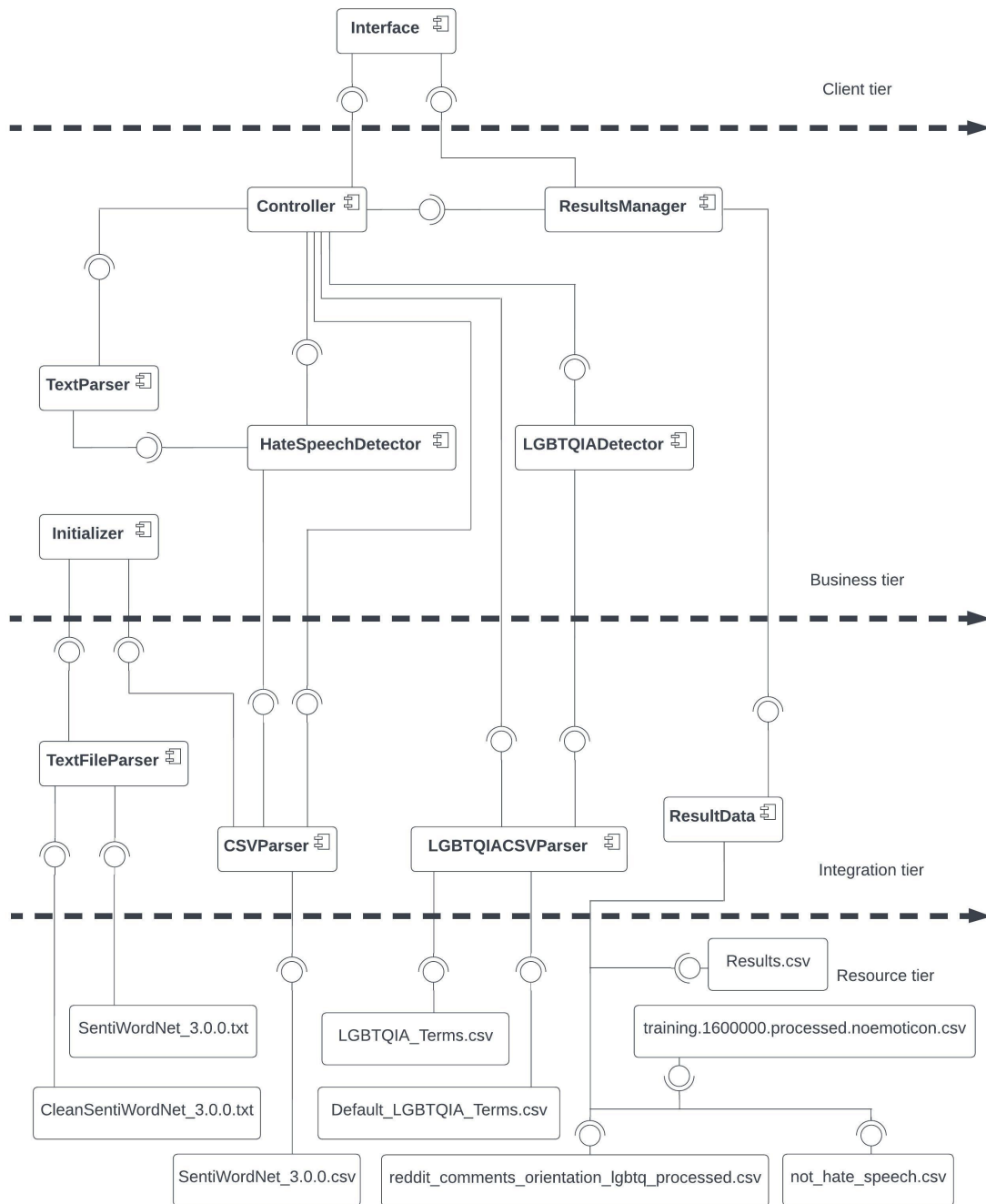


Figure 5.2: UML Component Diagram

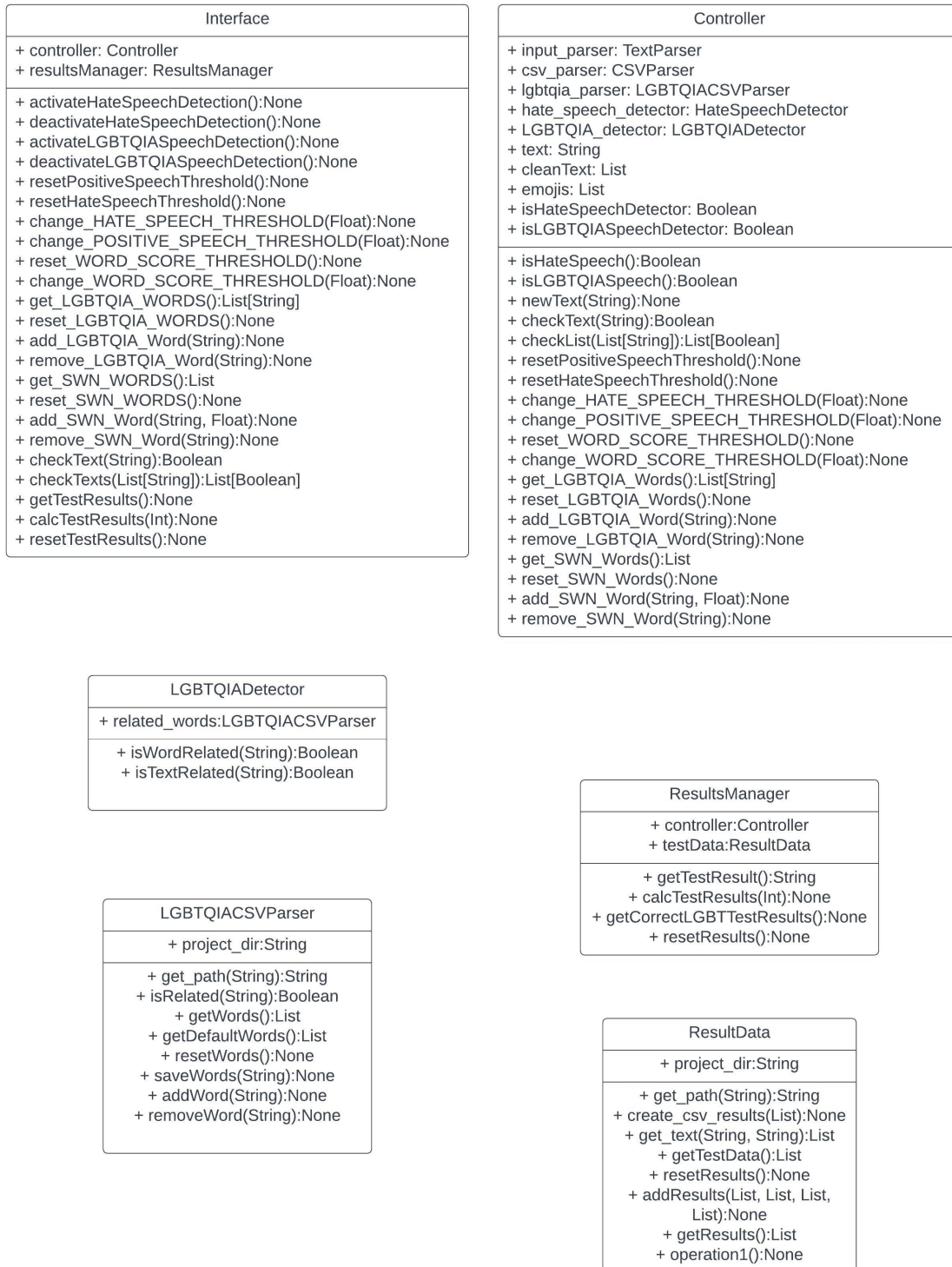


Figure 5.3: Class Diagram 1

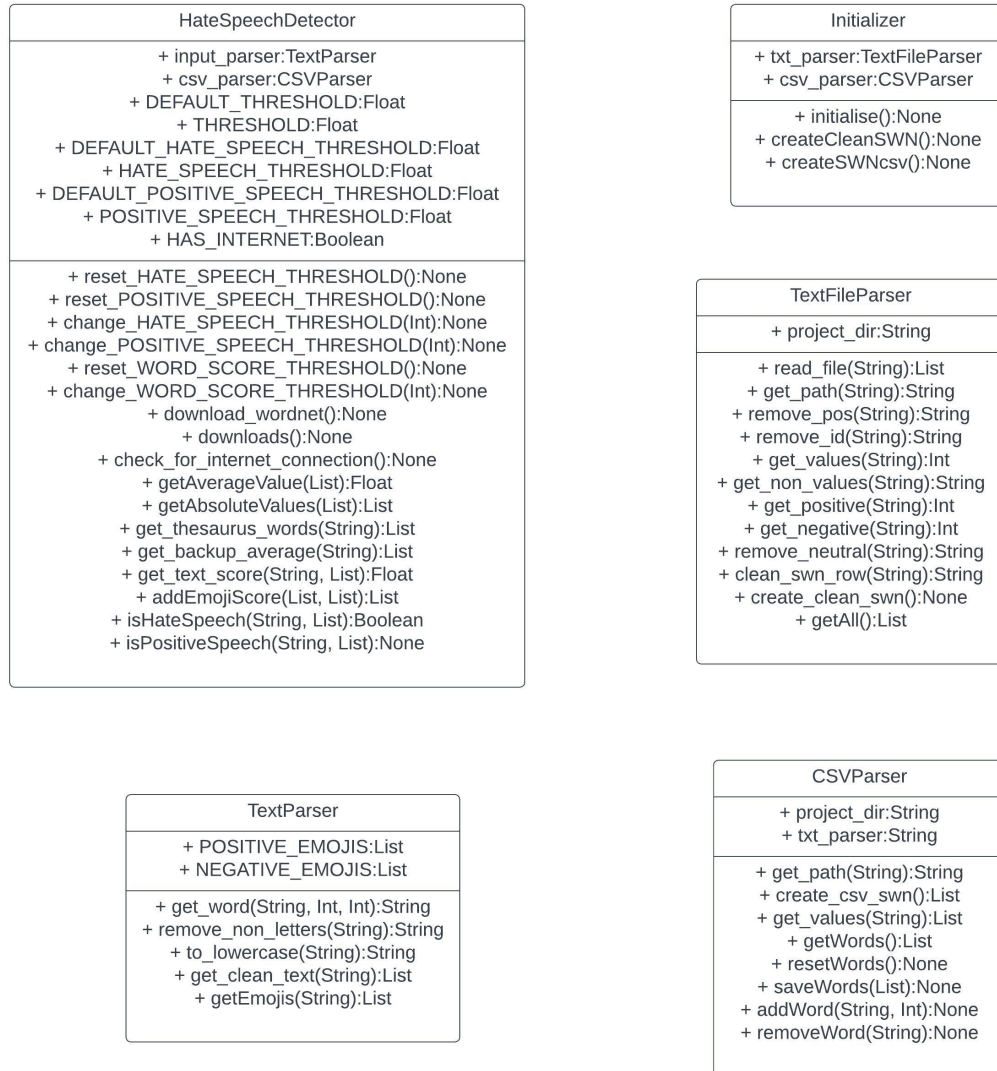


Figure 5.4: Class Diagram 2

Chapter 6

Implementation

The first step of implementation was to decide what language the system will be written in. Python was chosen because it is an easy-to-use language that can be readable to viewers of the code, this is useful due to maintenance and further development of the system. Python is one of the most popular programming languages widely used.[25] This then led to the popular PyCharm integrated development environment (IDE) developed by JetBrains being used. This assisted with the development process by assisting with syntax, error detection and other development tools.

Deciding on a software development process is a beneficial choice to make when developing a system because it can streamline development, allowing the developers to increase productivity by keeping to a plan. The code quality can also be improved by some practices such as test-driven development, which consists of writing the tests for code that has not yet been developed and then the goal is to write the code afterwards to pass the tests.[26] There are other development processes to adopt such as the Waterfall Model.[18] They all have their own advantages and disadvantages, taking these into consideration, the agile development process was chosen for the system. This was done because, with the use of a kanban board, a method to break up tasks into manageable sections.[21] This means scrums were easily planned and worked through, allowing a constant flow of development, making iterative progress and developing the system in a modular way.

The preceding sections will go through the development phases in chronological order. These sections will end with a description of the testing phase and some issues that arose when testing. When the letter 'R' is followed by numbers in square brackets, they refer to the requirement

with that ID.

6.1 Text parser

The first module that was developed was the text parser. The main goal of this section was to reformat any text into a list of individual words that created the text, as well as produce a list that represents the emojis used within the text. This would create the standard format that the rest of the system would assume the text would be in. This allows R[1.3] to be met.

The first step was to remove all non-letter characters. This was done due to these characters corrupting the rest of the word they might be attached to. This would later cause difficulties in the analysis and comparison of words. This was done by analysing every character that created the text. If the character was an alphabetical character or if it was a blank space then it was saved, otherwise, it was discarded.

The next step was to convert all the remaining characters to lowercase. This was done with the built-in python method `'lower()'`.

The last step to acquiring the standard format for text was to split up the words and add them to a list. This was developed by going through each character of the text, keeping track of two indexes, front and back, using these indexes and the locations of blank spaces, each word is identifiable. These words are added to a list to be returned, ready for analysis.

Removing all the non-letters from the given text does have a downside, you can lose emojis due to them sometimes being created from these non-letter characters. To counter this issue, the system uses the original format of the text to locate all the positive and negative emojis. This is done by cross-referencing the stored list of all positive and negative emojis[1][6] and checking if they are contained in the text R[1.6]. What is returned is not a list of the exact emojis but instead, a list that records how many positive and negative emojis were found. This is done because other parts of the program should not need to know what exact emojis were found, but instead how many positive and negative emojis were found.

6.2 Reformatting SentiWordNet

SentiWordNet 3.0.0[2] is a database that lists synsets with positive and negative values. Synsets are similar to synonyms, so similar-meaning words are grouped together. The database of synsets that SentiWordNet 3.0.0 has gathered is over 117,000 entries. This dataset will be at the core of the hate speech detection algorithm so it is in the best interest to make this dataset as efficient as possible without losing any quality from the data R[2.1].

All entries that had a positive value equal to the negative value were removed. This is because the overall value would be 0. This also included all entries that had the value 0 for both positive and negative. With this one improvement, the number of entries was reduced to 27,873.

Next, all of the redundant information for each entry was removed, this included but was not limited to, comments, ID and synset definition.

The last format change was the decision to combine the positive and negative scores. Since the evaluation of the words will remove context, it would be unable to consider what value of the two to use. Thus, combining them now will just simplify later steps due to only needing to work with 1 value per synset.

After all the changes, this left just the value for the synset followed by all the words contained in the synset. All of these changes were added to a new .txt file called 'CleanSentiWordNet_3.0.0.txt' and then later converted to a .csv file called 'SentiWordNet_3.0.0.csv' for easier access and maintainability. One method call given a word will return a list of all the values for the word given the different synsets it is found in.

6.3 Hate Speech Detection

The hate speech detector is given the preprocessed text and a representation of emojis. With this information, it will be able to classify the text being overall positive or negative, thus achieving R[1.1].

The first step will be to iterate the list of words to calculate a single score for each word. This is done by retrieving the list of values SentiWordNet (SWN) provides by using the CSVParser class. A mean average value is then created with the numbers. If this number is not '0' then the value is added to a list to contain all the values for the words within the text. If the average

value obtained is '0', then a thesaurus is used to get similar words to the word.[17] These synonyms, provided by NLTK WordNet[14], will then be looked up in SWN for values. The average for the words' overall scores is then returned to be the backup value. However, the path of getting backup value only happens if SWN values actually contain values. For example, if the word 'ABCD' returns no values from SWN then no thesaurus words are looked for. However, if the word 'WXYZ' have the values returned as '1' and '-1'. This would create the average of '0' so the thesaurus words would be looked up. This is done for two reasons. First, many words will not appear in SWN because they are either too neutral or they are just connective words, so they were already removed from SWN. This means that finding similar words to these can distort the original meaning behind the words. Compared to finding synonyms for words that did have values, this is done because they already had values, suggesting that they had useful meaning and this score is worth finding. The second reason is for performance R[2.1], many words do not have values in SWN, which means that finding the synonyms for all of these words is computationally expensive and should be avoided when possible.

The next step is to calculate the absolute values. This is simplifying the scores of the words into either '1' or '-1'. This is done by looking at each score and deciding if they reach the threshold above or below '0'. If they reach the threshold then the score is changed to the new value '1' or '-1', depending on what direction the threshold was met. For example, if the threshold is '0.2', then the values greater than '0.2' will turn to '1' and values less than '-0.2' will be '-1'. However, in this implementation, when a negative value becomes '-1' it actually creates two '-1' values. This will create a bias to give negative words more influence than positive words. All values between the thresholds will be removed. This is done to have more control over how extreme the words must be to be considered. This consequently identifies which words are positive, negative or if removed, neutral, achieving R[1.4].

The next step is to calculate the scores for the emojis. Emojis are a clear indicator of what the full text is meaning. Therefore, each positive emoji is worth three '1' values and each negative emoji is worth three '-1' values. Due to the emojis in the system all having an explicit meaning behind them, it is safe to give them these extreme values because when they are detected, they will act more as a fail-safe, to help guarantee the results.

The scores of all the words combined with the scores of the emojis are combined to create an overall text average score. For text to be considered hate speech it must be less than the 'hate

speech threshold' value. For text to be considered positive it must be more than the 'positive speech threshold' value.

6.4 LGBTQIA+ Detection

The LGBTQIA+ detection module will be responsible for detecting whether or not the text is targeted towards the LGBTQIA+ community. This completes R[1.2]. It was implemented by cross-referencing each word within some text with the known LGBTQIA+ related words.[10][3] If a match has been found then it assumes that the text is related towards the LGBTQIA+ community R[1.5]. The list of words is stored on the LGBTQIA_Terms.csv file.

The use of a single .csv file allows the system to be interchanged with other .csv files to adapt the whole program to detect hate speech towards other minorities or groups of people.

6.5 The Controller

The controller manages most of the other modules so the system functions as intended. For each new input of text, there are 3 main steps. First, the text is run through the text parser. This will reformat the text into the desired state. Secondly, the text and the list of emojis will be passed into the hate speech detector. This will return the results, whether or not the text is considered hate speech or not. Finally, the text is passed into the LGBTQIA+ detector. This will also return a result, now determining if the text is LGBTQIA+ related. Combining the 2 results, the text can be identified as hate speech towards the LGBTQIA+ community, returning True. If it is not hate speech towards the LGBTQIA+ community then it will return false. This process can also be done with a list of text, producing a list of results.

6.6 Analysis of Test Data

The system was designed to include test data. This is because it will allow any user to test their modifications of the system immediately. This section will also reach R[3.8], R[3.9] and R[3.10].

The first step to this is to acquire suitable test data. This data must already be classified so there is a target for the system to reach. For hate speech towards the LGBTQIA+ community examples, this came from 'RedditBias: A Real-World Resource for Bias Evaluation and

Debiasing of Conversational Language Models’.[4] This provides examples of Reddit comments that are hate speech towards different groups of people. One of which had the orientation category, this dataset will be used for the test data. This data set is 9930 examples in size and it is stored in the .csv file called ‘reddit_comments_orientation_lgbtq-processed.csv’. The source for the positive text examples is from ‘Twitter Sentiment Classification using Distant Supervision’[11]. Unfortunately, the data set does not include a mention of what examples are LGBTQIA+ related and what are not, this means it cannot be assumed either way and instead just classified as positive test data. The original dataset had 1.6 million examples with 800,000 being positive examples and 800,000 being negative examples. I reduced this dataset to 10,000 positive examples, this was done so the number of entries was closely matching the hate speech examples and any more than 10,000 would be unnecessary. 10,000 entries alone give the user plenty of examples to test on. These examples are stored on the .csv file called ‘not_hate_speech.csv’. This means that the total test data is about 20,000 different examples. It was possible to obtain hate speech examples that have no LGBTQIA+ relation however, it was decided that these examples would be unnecessary for testing due to hate speech detection on its own can still be analysed without the LGBTQIA+ relation. This is because all LGBTQIA+ hate speech is just a subset of all hate speech.

Due to the time needed to compute results, the system allows the user to specify how many results to compute at once. This provides control over the computational power given to the testing phase. Computing the results in smaller sections will allow the user to control how accurate the final results will be. It is recommended for the user to compute less than 100 results at a time for quick calculations. The results are only saved after the given number of results has been calculated. This means that multiple groups of 100s can be queued and if the process is stopped during the process, the batches that were completed are saved. A progress counter is displayed in the terminal so the user knows how many examples in the processing batch have been calculated.

Each call to calculate results from the test data will provide results for the ‘LGBTQIA+ hate speech’ and the ‘non-hate speech’ equally. For example, if want to calculate 100 test results, 50 new LGBTQIA+ hate speech examples and 50 new non-hate speech examples will be calculated. The results will be stored in ‘Results.csv’ in the format of ID, target class, predicted class then text. Running the calculation of results multiple times will create more results for test data that have yet to be analysed. This is beneficial because repeating calculations for examples

already seen is a waste of resources. The user can reset the results file. This can be useful for two scenarios. The first is if the results file was corrupted then a new version can be made without the user knowing how to make the file. Secondly, if the user would like to test the performance of the algorithm on new parameters or methods then they are able to. When the results are requested from the user, the information will be displayed in the terminal. This is done so the user can easily understand the information without having to do extra calculations or computations.

6.7 Interface

The interface is a class within the system, the idea is that the user will only ever directly call this class to interact with the rest of the system. The object itself does not do much computation however, it gives the user the ability to view all the different ways to interact from one place R[2.3].

There are multiple methods the user can call. One of them called 'checkText' requires text as a String and it will return True if hate speech towards the LGBTQIA+ community was found, and False if it was not found R[3.1]. The method named 'checkTexts' acts in a similar way to 'checkText' however the input must be a list of Strings and it returns a list of answers R[3.2].

6.8 Additional Features

Additional features were added near the end of the development process. This is done because many of these features were low in priority due to them not being part of the core components of the system. Also, many of these features relied upon the development of other modules first to work. Due to the modular design of the system, the implementation of the features was easily achieved.

The method found in the Interface class called 'activateHateSpeechDetection' will turn the hate speech detection on whilst the 'deactivateHateSpeechDetection' will turn it off R[3.3]. Similarly, the method 'activateLGBTQIASpeechDetection' will turn the LGBTQIA+ detection on whilst the 'deactivateLGBTQIASpeechDetection' will turn it off R[3.4].

The methods that can be found in the interface called 'change_HATE_SPEECH_THRESHOLD' and 'change_POSITIVE_SPEECH_THRESHOLD' will take in a number requested to be be-

tween '0' and '1'. The input will change the system's internal values to change the requirements for text to be classified as positive or negative. R[3.5] R[3.6]. There is also the option to use 'resetPositiveSpeechThreshold' and 'resetHateSpeechThreshold', this will reset the values to the original number. The user can also call 'change_WORD_SCORE_THRESHOLD' with a number, recommended to be between '0' and '1', to change the threshold a word must reach to be considered positive or negative R[3.7]. This value can also be reset to the default with 'reset_WORD_SCORE_THRESHOLD'.

The ability of the user to be able to customize the database of words with their corresponding values is a useful feature. Through the interface, the user can be returned a list of all the values with the corresponding words with 'get_SWN_WORDS'. A new word and value can be added with 'add_SWN_Word', and words can also be removed entirely from the database with 'remove_SWN_Word'. After all the modifications, the user also has the ability to reset all the words and values with 'reset_SWN_WORDS'. This allows the user to not worry about experimenting with new words and values because they can always start fresh. These features achieve R[3.11], R[3.12], R[3.13] and R[3.14].

The user can also request a list of all the LGBTQIA+ related words through the interface with 'get_LGBTQIA_WORDS' R[3.15]. The requirements R[3.16], R[3.17] and R[3.18] can also be achieved by the interface methods 'add_LGBTQIA_Word', 'remove_LGBTQIA_Word' and 'reset_LGBTQIA_WORDS'.

The details on the implementation were simple, so refer to the source code if more information is required.

6.9 Testing

Testing was achieved by using unit tests. These tests will call particular parts of the code with set parameters and then record the results given. These results are then compared against the expected results obtained. These comparisons will determine if the tests pass or fail. The tests cover 95% of the entire system. Some parts of the system are difficult to test, this is because they might rewrite files and if the users have saved results or modifications within the file, then they would be deleted.

Chapter 7

Legal, Social, Ethical and Professional Issues

7.1 Different English speaking cultures

People from different cultures could use the program to analyse text. When the tool comes to a conclusion on how to classify the text for example if it classified the text 'A' as something is hate speech towards the LGBTQIA+ community when a matter of fact, in their culture, the words from 'A' might have different meanings and thus would actually be considered not offensive due to the context of the cultural difference. This means that the tool might work in one place however, it might not be as accurate when used in other places.

7.2 Miss-classification

Due to misclassification, for example, identifying hate speech that is not actually hate speech can be damaging. This is because if this system or others like it are used with no manual observation within a social media system or other systems that have a chat/text input, then it could cause the owner of the miss-classified text to get restricted on the platform or even banned whilst doing nothing wrong.

7.3 British Computer Society's Code of Conduct

In the British Computer Society's Code of Conduct[23] states "only undertake to do work or provide a service that is within your professional competence"[23]. This applies to the project because my knowledge of Artificial Intelligence was not as strong at the start of the project. This meant that I should not attempt to develop a system with methods I would not understand. Also, it states that you must "respect and value alternative viewpoints and seek, accept and offer honest criticisms of work"[23]. This is key when evaluating methods during the Background chapter as well as reviewing results in the Results and Evaluation chapter. The code of conduct "avoid injuring others, their property, reputation, or employment by false or malicious or negligent action or inaction"[23] relates to the misclassification as stated above. This point will be taken into consideration in the Conclusion chapter. In the Results and Evaluation chapter "NOT misrepresent or withhold information on the performance of products"[23] is very important. The system must be represented in a fair and unbiased perspective so any potential user has knowledge of the system and its performance.

Chapter 8

Results and Evaluation

8.1 Evaluation

All of the functional requirements were achieved. The non-functional requirements can not be easily determined if they are completed or not however, the system did take them into consideration during the design and implementation phases. The user requirements between R[3.1] to R[3.18] have been achieved. The requirements R[3.19] and R[3.20] were not completed. This means that the user cannot save or load previous settings they have used. These were in the low-priority group for requirements so it was not detrimental for these features not to be completed.

8.2 Results

When analysing the data the term true positive (TP) will refer to an item when it is classified correctly for the class being analysed. A false positive (FP) is when an item is classified as the class being analysed however, it is not actually that class. A false negative (FN) is when an item has failed to be classified as the class being analysed, instead, they were classified as something else. Finally, a true negative (TN) is when an item is not classified as the class being analysed, and this classification was true.

8.2.1 Precision

The precision of classification shows the rate when the classification is right when the classification is positive.[12]

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad (8.1)$$

8.2.2 Recall

The recall of classification shows how many within the class were actually identified correctly.[13]

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negative} \quad (8.2)$$

8.2.3 Hate Speech Detection

Using hate speech as the target class the precision and recall values are as follows.

$$Precision = \frac{1222}{1222 + 480} = 0.72 \quad (8.3)$$

$$Recall = \frac{1222}{1222 + 1278} = 0.49 \quad (8.4)$$

8.2.4 LGBTQIA+ Detection

Using LGBTQIA+ speech as the target class the recall value is as follows. The precision could not be calculated due to no true negative tests.

$$Recall = \frac{2500}{2500 + 0} = 1.0 \quad (8.5)$$

8.2.5 LGBTQIA+ Hate Speech Detection

Using LGBTQIA+ hate speech as the target class the precision and recall values are as follows.

$$Precision = \frac{1222}{1222 + 110} = 0.92 \quad (8.6)$$

$$Recall = \frac{1222}{1222 + 1278} = 0.49 \quad (8.7)$$

8.2.6 Results Analysis

These values were from the results of 5000 tests. It shows that the system is hate speech has good precision. However, the recall of hate speech and LGBTQIA+ related hate speech is low and at the rate the system is, it is about the same as guessing. This could be for many reasons such as insufficient databases used in classification or the algorithm itself having flaws. To

understand more, the scoring system must be evaluated in more detail. It is worth noting the precision of LGBTQIA+ hate speech and LGBTQIA+ related content with a value of 0.92 and 1.0 respectively. This shows that if an item is classified as hate speech towards the LGBTQIA+ community or classified as LGBTQIA+ related, then there is a very high accuracy that the classification is correct.

True Positive Example

The sentence "gay people are mentally ill" was classified as LGBTQIA+ related hate speech and this was correct. The values for each word within the sentence are as followed:

gay : ['0.5', '0.25', '0.625', '0.125']
people : []
are : []
mentally : ['0.125']
ill : ['-0.5', '-0.875', '-0.75', '-0.125', '-0.625', '-0.25', '-0.75', '-0.5', '-0.625']

The final non '0' values for each word are:

gay : 0.375
mentally : 0.125
ill : -0.5555555555555556

This creates an overall score of '-1', classifying the text as hate speech. The LGBTQIA+ related word within the sentence was identified as 'gay', this means that the text was classified as LGBTQIA+ related.

True Negative Example

The sentence "Denise, I love you. " was classified as not hate speech and this was correct. The values for each word within the sentence are as followed:

denise : []
i : ['-0.25']
love : ['0.375', '0.25', '0.625', '0.125', '0.25', '0.5', '0.625', '1.0']
you : []

The final non '0' values for each word:

i : -0.25

love : 0.46875

This creates an overall score of 0, classifying the text as not hate speech. There were no LGBTQIA+ related words within the sentence, which means that the text was classified as not LGBTQIA+ related.

False Negative Example

The sentence "many gays are flamboyant and attention-seeking" was classified as not hate speech when it actually was an example of LGBTQIA+ related hate speech. The values for each word within the sentence are as followed:

many : []

gays : []

are : []

flamboyant : ['-0.125', '0.5']

and : []

attentionseeking : []

The final non '0' values for each word:

flamboyant : 0.1875

This creates an overall score of 0, classifying the text as not hate speech. This was caused by the word 'flamboyant' being positive as well as no negative words were found. The LGBTQIA+ related word within the sentence was identified as 'gays', this means that the text was classified as LGBTQIA+ related.

False Positive Example

The sentence "trying to figure this thing out" was classified as hate speech when it actually was an example of non-hate speech. The values for each word within the sentence are as followed:

trying : ['-0.625', '-0.5']

to : []

figure : ['-0.375', '0.25']

this : []

thing : ['-0.125', '-0.125']

out : ['-0.125', '-0.5', '-0.875', '-0.75']

The final non '0' values for each word:

trying : -0.5625

figure : -0.0625

thing : -0.125

out : -0.5625

This creates an overall score of -1, classifying the text as hate speech. This was caused by the words 'trying', 'figure', 'thing' and 'out' all having negative scores. Even though these words do not seem negative, they have resulted in the text being classified as such. There were no LGBTQIA+ related words within the sentence, which means that the text was classified as not LGBTQIA+ related.

Analysis

The misclassification of hate speech can be seen when words that seem to not have much meaning behind them actually carry a large score. This causes the average of the text to be weighted more on one end and this can cause misclassification. Increasing the size of the database would improve classifying words that actually do hold meaning however, the problem of words with scores when they should not have would not improve. Therefore, a new database with a more restricted and precise scoring of words would improve the hate speech detection system.

8.3 Related Word

In 'Lexicon-Based Sentiment Analysis in the Social Web'[17] they were classifying text into positive or negative using a lexicon-based approach. They achieved 92% accuracy for the classification of positive and negative text. These results are more accurate than the system we made with an accuracy of 49% for positive and negative classification.

In 'Detecting Harmful Online Conversational Content towards LGBTQIA+ Individuals'[8] they were using out-of-the-box machine learning models. They used five different models and the model with the highest precision they obtained was 0.94 and the highest recall was 0.97. The recall result was also higher than the obtained result from our system at 0.49. However, the precision was similar with the system producing a 0.92.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

The benefit of not using training data with the lexicon-based method has been useful in the implementation phase. However, this approach still depends on a lot of other data such as the individual word values and the LGBTQIA+ related words. The word values file can be kept the same if the program is developed to target a different minority group however, the LGBTQIA+ related words will have to change. This would not be a difficult change because this file does not need to be very large. So this approach would be easier to adapt than many machine learning approaches using training data. However, one issue with this that was discovered in the development was that the system should be tested to identify the performance and this needs test data. This means that both lexicon-based and machine learning do need some sort of pre-classified data thus, removing one major advantage to the lexicon-based method.

From previous testing, the ability to change the parameters within the system to make the algorithms more or less strict has been a useful feature to use. This customizability would not be easily obtainable in other machine learning methods.

However, the system's overall hate speech accuracy is low and due to the sensitivity of the topic, this is a major factor when making conclusions about the system. Therefore, the tool in its current form is not recommended for use in other systems.

9.2 Future Work

Since the tool does provide many useful features, future development should be considered. Improving the databases could increase the accuracy of the system thus making it a useful tool. Analysing the current algorithms implemented and then improving them could also increase accuracy as well as improve performance. Creating a more interactive interface could make the tool easier for users to use. This could also allow development for more features whilst keeping the system simple to use.

Finally, the development of a similar system with a machine-learning approach could be worked on. This could then be used to classify the texts more accurately. Then the current tool could use these results, already knowing that they are LGBTQIA+ targeted hate speech, to then find the 'value' given to each text. This can then let the user control the 'intensity' of the detection to their desired level.

References

- [1] Muhammad Zubair Asghar, Aurangzeb Khan, Shakeel Ahmad, Maria Qasim, and Imran Ali Khan. Partial list of positive and negative emoticons., 2 2017.
- [2] Stefano Baccianella, Andrea Esuli, and Fabrizio Sebastiani. SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining. In *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10)*, Valletta, Malta, May 2010. European Language Resources Association (ELRA).
- [3] Heather Barbour. <https://blog.ongig.com/diversity-and-inclusion/lgbt-words/>, 2023.
- [4] Soumya Barikeri, Anne Lauscher, Ivan Vulić, and Goran Glavaš. Reddittbias: A real-world resource for bias evaluation and debiasing of conversational language models. *arXiv preprint arXiv:2106.03521*, 2021.
- [5] Aline Bessa. <https://www.knime.com/blog/lexicon-based-sentiment-analysis>, 2023.
- [6] Albert Bifet, Geoffrey Holmes, and Bernhard Pfahringer. Moa-tweetreader: Real-time analysis in twitter streaming data. In *Discovery Science - 14th International Conference, DS 2011, Espoo, Finland, October 5-7, 2011. Proceedings*, pages 46–60, 01 2011.
- [7] Dalton Conley. Wired for distraction: Kids and social media. *Time Magazine*, 2011.
- [8] Jamell Dacon, Harry Shomer, Shaylynn Crum-Dacon, and Jiliang Tang. Detecting harmful online conversational content towards lgbtqia+ individuals. *arXiv preprint arXiv:2207.10032*, 2022.
- [9] Oxford Advanced Learner’s Dictionary. <https://www.oxfordlearnersdictionaries.com/definition/english/hate-speech>, 2023.

- [10] We Are Family. <https://wearefamilycharleston.org/lgbt-glossary-az>, 2023.
- [11] Alec Go, Richa Bhayani, and Lei Huang. Twitter sentiment classification using distant supervision. *CS224N Project Report, Stanford*, 1(12):2009, 2009.
- [12] Google. <https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall>, 2023.
- [13] Google. <https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall>, 2023.
- [14] Koray Tuğberk GÜBÜR. <https://www.holisticseo.digital/python-seo/nltk/wordnet>, 2023.
- [15] IBM. <https://www.ibm.com/topics/machine-learning>, 2023.
- [16] java T point. <https://www.javatpoint.com/factory-method-design-pattern>, 2023.
- [17] Fazal Masud Kundi, Aurangzeb Khan, Shakeel Ahmad, and Muhammad Zubair Asghar. Lexicon-based sentiment analysis in the social web. *Journal of Basic and Applied Scientific Research*, 4(6):238–48, 2014.
- [18] Ben Lutkevich. <https://www.techtarget.com/searchsoftwarequality/definition/waterfall-model>, 2023.
- [19] Cyprien Nielly. <https://towardsdatascience.com/can-we-let-algorithm-take-decisions-we-cannot-explain-a4e8e51e2060>, 2023.
- [20] Jonathan A Obar and Steven S Wildman. Social media definition and the governance challenge-an introduction to the special issue. *Obar, JA and Wildman, S.(2015). Social media definition and the governance challenge: An introduction to the special issue. Telecommunications policy*, 39(9):745–750, 2015.
- [21] Isaac Sacolick. <https://www.infoworld.com/article/3237508/what-is-agile-methodology-modern-software-development-explained.html>, 2023.
- [22] Isaac Computer Science. https://isaaccomputerscience.org/concepts/soft_design_modular?examBoard=all&stage=all, 2023.
- [23] British Computer Society. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>, 2023.

- [24] Joseph Spinelli. https://medium.com/@joespinelli_6190/mvc-model-view-controller-ef878e2fd6f5, 2023.
- [25] TECHDUDE. <https://www.aclti.com/en/blog/the-most-popular-language-is-python>, 2023.
- [26] Jash Unadkat. <https://www.browserstack.com/guide/what-is-test-driven-development>, 2023.

Appendix A

Extra Information

A.1 Tables, proofs, graphs, test cases, ...

The appendices contain information that is peripheral to the main body of the report. Information typically included in the Appendix are things like tables, proofs, graphs, test cases or any other material that would break up the theme of the text if it appeared in the body of the report. It is necessary to include your source code listings in an appendix that is separate from the body of your written report (see the information on Program Listings below).

Appendix B

Source Code

B.1 Instructions

Complete source code listings must be submitted as an appendix to the report. The project source codes are usually spread out over several files/units. You should try to help the reader to navigate through your source code by providing a “table of contents” (titles of these files/units and one line descriptions). The first page of the program listings folder must contain the following statement certifying the work as your own: “I verify that I am the sole author of the programs contained in this folder, except where explicitly stated to the contrary”. Your (typed) signature and the date should follow this statement.

All work on programs must stop once the code is submitted to KEATS. You are required to keep safely several copies of this version of the program and you must use one of these copies in the project examination. Your examiners may ask to see the last-modified dates of your program files, and may ask you to demonstrate that the program files you use in the project examination are identical to the program files you have uploaded to KEATS. Any attempt to demonstrate code that is not included in your submitted source listings is an attempt to cheat; any such attempt will be reported to the KCL Misconduct Committee.

You may find it easier to firstly generate a PDF of your source code using a text editor and then merge it to the end of your report. There are many free tools available that allow you to merge PDF files.